

Практическая работа №1

Пользовательские запросы в веб-приложении (DevTools, HTTP, REST, JSON, Postman)

1.Теория:

1.1. Что такое DevTools

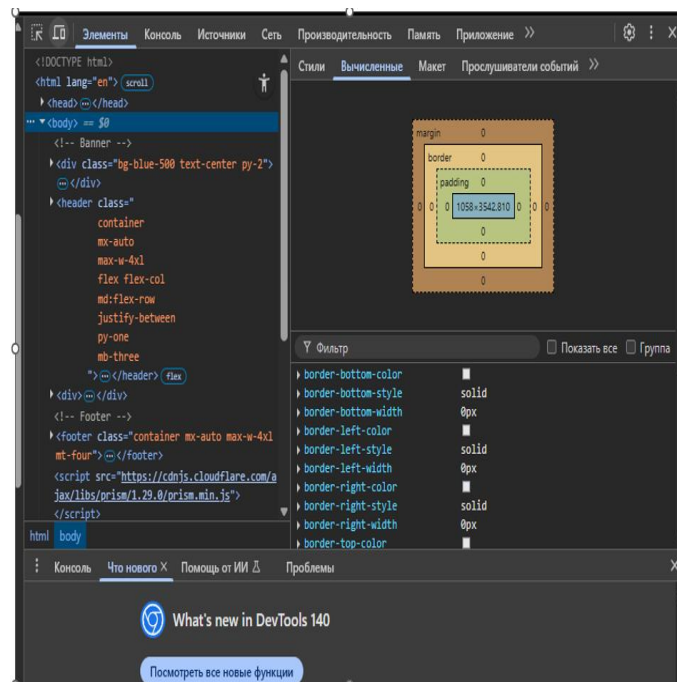
DevTools (Developer Tools) — встроенные инструменты разработчика браузера (Chrome, Firefox, Edge, Safari).

Открытие: **F12**.

Основные панели:

- **Elements** — DOM и визуальная структура страницы.
- **Console** — ошибки, логи, выполнение JS.
- **Network** — анализ запросов, коды, заголовки, время.
- **Performance** — измерение скорости работы страницы.
- **Device mode** — проверка адаптивности интерфейса.
- **Security** — HTTPS, сертификаты.
- **Memory** — утечки памяти и использование ресурсов.

Пример интерфейса DevTools:



1.2. Пользовательские запросы

Пользовательские запросы — это **HTTP-запросы** (GET, POST и др.), которые браузер отправляет при работе с веб-приложением.

Основные характеристики запроса:

- метод (GET/POST/PUT/DELETE/...)
- URL и параметры
- заголовки (headers)
- тело запроса / ответа
- время выполнения
- код ответа

Оптимизация запросов:

- уменьшение количества обращений;
- сокращение объёма передаваемых данных;
- кэширование;
- пагинация и фильтрация;
- использование правильных HTTP-методов и заголовков.

Основные HTTP-методы

Метод	CRUD	Назначение	Особенности	Пример
GET	Read	Получение данных	Параметры в URL, кэшируемый, не меняет данные	/users?id=5
POST	Create	Создание ресурса	Данные в теле, изменяет состояние	Регистрация, заказ
PUT	Update	Полное обновление	Заменяет весь объект	Обновить профиль
PATCH	Update	Частичное обновление	Меняет только указанные поля	Сменить только email
DELETE	Delete	Удаление ресурса	Обычно 200 OK или 204 No Content	Удалить товар
HEAD	—	Только заголовки	Проверка наличия / размера / даты	Проверка Last-Modified

1.3. Коды ответов

- 1xx — информационные (например, 100 Continue).
- 2xx — успешные (200 OK, 201 Created, 204 No Content).
- 3xx — перенаправления (301, 302, 304 Not Modified).
- 4xx — ошибки клиента (400, 401, 403, 404).
- 5xx — ошибки сервера (500, 502, 503, 504).

1.4. REST API

REST — архитектурный стиль взаимодействия с ресурсами по HTTP.

Принципы:

- клиент-сервер;
- отсутствие состояния (stateless);
- единообразный интерфейс;
- работа с ресурсами (уникальные URL);

- часто используется формат **JSON**.

1.5. JSON

JSON — лёгкий текстовый формат обмена данными (объекты, массивы, строки, числа, boolean, null).

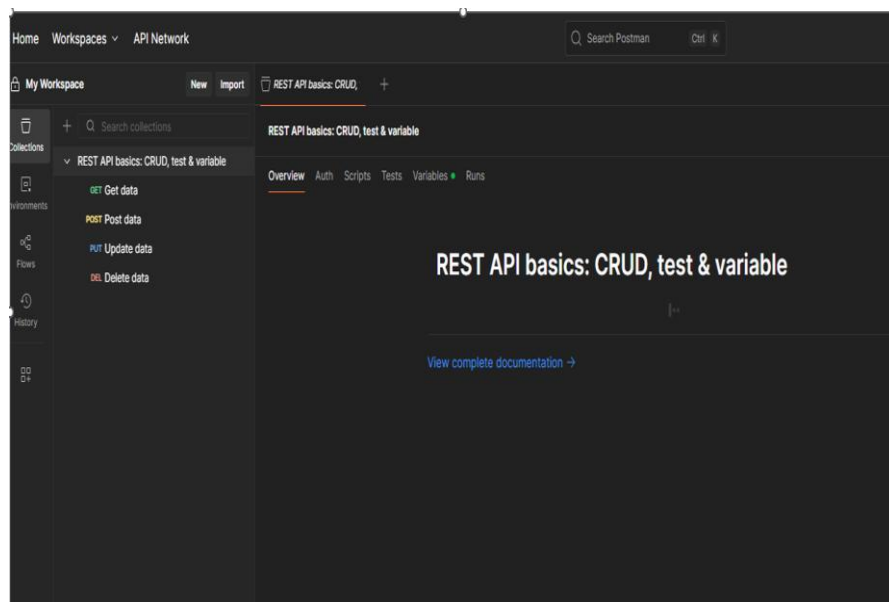
Пример ответа:

```
{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere...",
  "body": "quia et suscipit..."
}
```

1.6. Postman

Postman — инструмент для работы с API:

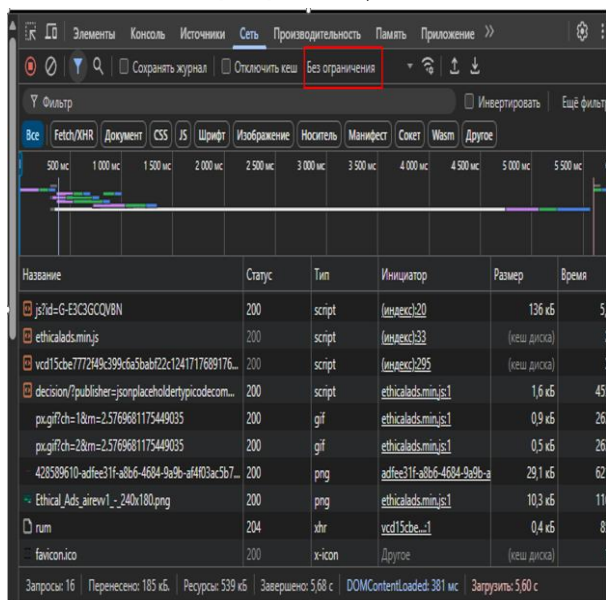
- создание и отправка запросов;
- анализ ответов;
- сохранение коллекций;
- автотесты;
- совместная работа.



2. Задание

2.1. Требуемые инструменты

- браузер (Chrome / Firefox / Edge / Яндекс.Браузер);
- DevTools → вкладка **Network** (режим Online, без ограничения скорости или с 4G-профилем).



2.2. Первичный анализ запросов

1. Откройте DevTools (**F12**) → вкладка **Network**.
2. Загрузите страницу:

<https://jsonplaceholder.typicode.com/posts>

1. Зафиксируйте в таблице: метод, URL, код ответа, размер данных, время загрузки.

метод	GET
URL	https://jsonplaceholder.typicode.com/posts
Код ответа	200
Размер данных	7.9 KB
время	232 ms

2. Сделайте скриншот списка запросов.

Name	Status	Type	Initiator	Size	Time
 posts	200	docume...	Other	7.9 kB	232 ms
 favicon.ico	200	x-icon	Other	0.8 kB	192 ms

2 requests

8.8 kB transferred

27.8 kB resources

Finish: 461 ms

DOMContentLoaded: 255 ms

2.3. Детальный разбор одного запроса

Выберите любой запрос и опишите:

- метод;
- URL и параметры;

- заголовки (Request Headers / Response Headers);
- тело ответа (Response / Preview);
- код ответа;
- время выполнения.

Сравните его с другим запросом и сделайте вывод.

✕ Headers Preview Response Initiator Timing	
▼ General	
Request URL	https://jsonplaceholder.typicode.com/posts
Request Method	GET
Status Code	● 200 OK
Remote Address	104.21.59.19:443
Referrer Policy	strict-origin-when-cross-origin
▼ Response headers	
Access-Control-Allow-Credentials	true
Age	13053
Alt-Svc	h3=":443"; ma=86400
Cache-Control	max-age=43200
Cf-Cache-Status	HIT
Cf-Ray	a38595bf6fd22aa5-ARN
Content-Encoding	gzip 
Content-Type	application/json; charset=utf-8
Date	Wed, 09 Sep 2026 10:42:13 GMT
Etag	W/"6b80-Ybsq/K6GwwqrYkAsFxqDXGC7DoM"
Expires	-1
Nel	{"report_to": "heroku-

2.4. Эксперименты с запросами

Выполните в консоли:

```
fetch("https://jsonplaceholder.typicode.com/posts/1")
  .then(res => res.json())
```

```
.then(data => console.log(data));
```

Проанализируйте запрос во вкладке **Network**. Сравните:

- `GET /posts` (все посты);
- `GET /posts/1` (один пост).

Сделайте выводы по:

- размеру ответа;
- времени загрузки;
- количеству записей.

	posts	posts/1
размер ответа	7.9 KB	1 KB
время загрузки	228 ms	250 ms
количество записей	100	1

2.5. Работа с параметрами

1. Откройте в браузере:

```
https://jsonplaceholder.typicode.com/posts?userId=1
```

1. Сравните с `https://jsonplaceholder.typicode.com/posts`:
2. сколько объектов вернулось;
3. размер ответа;
4. время загрузки.

Сделайте вывод, как параметры помогают **уменьшить объём данных**.

	posts/1	posts?usedid=1
объектов вернулось	1	10
размер ответа	1 KB	1.8 KB

время загрузки	256 ms	255 ms

2.6. Массовые запросы

Откройте несколько URL одновременно:

- `/posts/1`
- `/posts/2`
- `/posts/3`

Зафиксируйте:

- как меняется время отклика;
- как браузер параллелит запросы;
- нет ли избыточных запросов.

 1	200	fetch	index.html:10	1.0 kB	377 ms
 2	200	fetch	index.html:12	1.0 kB	491 ms
 3	200	fetch	index.html:14	1.0 kB	607 ms

2.7. Заголовки и ответы

1. Во вкладке **Headers** выпишите не менее 5 ключевых заголовков, например:
2. `Content-Type`
3. `Cache-Control`
4. `ETag`
5. `Date`
6. `Server`
7. Сравните вкладки **Preview** и **Response**:
8. чем они отличаются по представлению данных;
9. в каких случаях удобнее использовать Preview, а в каких — Response.

Content-Type	application/json; charset=utf-8	Указывает клиенту, в каком формате пришли
--------------	------------------------------------	--

		данные и в какой кодировке.
Cache-Control	max-age=43200	Управляет кешированием. Значение max-age=3600 говорит браузеру, что этот ресурс можно хранить в локальном кеше и не запрашивать с сервера повторно в течение 1 часа
ETag	W/"124-yiKdLzqO5gfBrJFrcdJ8Yq0LGnU"	Уникальный идентификатор версии ресурса
Date	Wed, 09 Sep 2026 11:25:00 GMT	дата и время создания HTTP-сообщения
Server	cloudflare	программное обеспечение веб-сервера

- **Preview** — когда нужно **быстро посмотреть суть** данных: структуру JSON, отрендеренный HTML или картинку. Удобно при тестировании API и верстке.
- **Response** — когда нужно увидеть **точный сырой ответ** сервера: проверить экранирование, спецсимволы, переносы строк, скопировать данные для отладки или найти скрытые ошибки в тексте.

2.8. Оптимизация запросов

Сравните ответы:

Запрос	Размер ответа	Время загрузки
/posts	7.8 KB	488 ms

/posts?userId=1	1.8 KB	397 ms
/posts/1	1 KB	377 ms

Предложите меры оптимизации:

- пагинация (ограничение количества записей на страницу);
- фильтрация (по `userId`, статусам и т.п.);
- уменьшение размера ответа (убрать лишние поля);
- кэширование на клиенте и сервере;
- использование правильных заголовков (`Cache-Control`, `ETag` и т.д.).

2.9. Работа с API через Postman

1. Перейдите на сайт:

<https://www.postman.com/>

1. Выполните:

2. `GET /posts`

3. `GET /posts/1`

Сравните ответы.

1. Выполните **POST** `/posts` с JSON-телом:

```
{
  "title": "Пост через Postman",
  "body": "Текст поста",
  "userId": 2
}
```

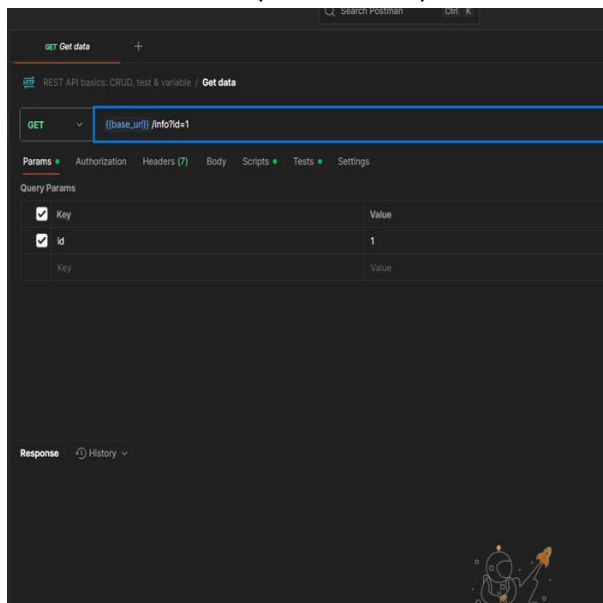
1. Выполните:

2. **PUT** `/posts/1` — полное обновление ресурса;

3. **PATCH** `/posts/1` — частичное обновление (изменить только один-два поля).

Объясните, чем отличаются результаты.

1. Выполните **DELETE** `/posts/1` и зафиксируйте:
2. код ответа;
3. тело ответа (если есть).



2.10. Сравнение DevTools и Postman

Сделайте таблицу или краткое сравнительное описание:

- **DevTools**
- анализ реальных запросов страницы;
- проверка производительности, кэширования;
- работа в контексте открытого сайта.
- **Postman**
- ручное конструирование запросов;
- сохранение коллекций;
- тесты и скрипты;
- работа с авторизацией и токенами.

Сделайте вывод: когда удобнее DevTools, а когда Postman.

3. Контрольные вопросы

1. Что такое DevTools и какие задачи решают вкладки Elements, Console и Network?
2. Как открыть DevTools и какие настройки вкладки Network важны для замеров производительности?

3. Перечислите основные части HTTP-запроса (метод, URL/параметры, заголовки, тело) и их назначение.
4. Чем отличаются методы GET, POST, PUT, PATCH и DELETE? Приведите по одному практическому примеру.
5. Для чего используются методы HEAD и OPTIONS и где встречается preflight-запрос?
6. Расскажите о группах кодов ответов (1xx–5xx) и приведите примеры для 200, 301/302, 304, 404, 500.
7. Что такое REST и как ресурс в REST представлен на уровне URL?
8. Какие типы данных поддерживает JSON? Приведите короткий пример JSON-объекта и массива.
9. В чём отличие вкладок Preview и Response в карточке запроса DevTools?
10. Какие заголовки чаще всего анализируются при оптимизации (например, Content-Type, Cache-Control, ETag, Authorization)?
11. Какие приёмы снижения нагрузки на сеть можно использовать для API: пагинация, фильтрация, кэширование и др.?
12. Чем работа с API в Postman отличается от анализа запросов в DevTools, и когда уместен каждый инструмент?

Чек-лист для самопроверки

Баллы	Критерии выполнения
7	Выполнены все пункты практики: – первичный анализ запросов (таблица + скриншоты); – детальный разбор + сравнение запросов; – работа с параметрами, массовые запросы, заголовки (5+); – таблица оптимизации (/posts, /posts?userId=1, /posts/1); – эксперименты в Postman (GET, POST, PUT, PATCH, DELETE); – сравнение DevTools и Postman; – все обязательные скриншоты (Network, Console, Postman);

	– есть итоговые выводы и рекомендации; – аккуратное оформление: таблицы, структурированные заголовки, выводы.
6	Выполнена полная практическая часть в DevTools (таблицы, скриншоты, сравнения); реализованы все виды запросов в Postman: GET, POST, PUT, PATCH, DELETE; есть сравнение DevTools и Postman; составлены рекомендации по оптимизации (5–7 мер); оформление есть, но структура слабее: не всегда удачно выбраны заголовки, таблицы оформлены неаккуратно.
5	Выполнены основные шаги анализа запросов: таблица с методами, URL, кодами ответа и временем; есть сравнение <code>/posts</code> и <code>/posts/1</code>, работа с параметрами; сделано 2–3 скриншота; разобраны заголовки и показаны примеры запросов с параметрами; примеры оптимизации приведены, но не оформлены отдельной таблицей; в Postman реализованы только GET/POST.
3–4	Краткое описание DevTools и HTTP-запросов; выполнен первичный анализ запросов (таблица: метод, URL, код ответа, размер, время); сделан хотя бы один скриншот из DevTools; есть разбор одного запроса, но без сравнений и выводов; JSON/REST API упомянуты, но Postman не использован или использован формально.
1–2	Даны лишь отдельные ответы без общей структуры;

	скриншоты отсутствуют или не соответствуют заданию; теоретическая часть слабо раскрыта, практическая часть выполнена формально.
0	Работа выполнена с помощью ИИ (автоматически сгенерированные ответы без личных скриншотов и анализа); отсутствует выполнение практических шагов, присутствует только копипаст из методических материалов.